

Парадигмы Software 1.0, 2.0 и 3.0

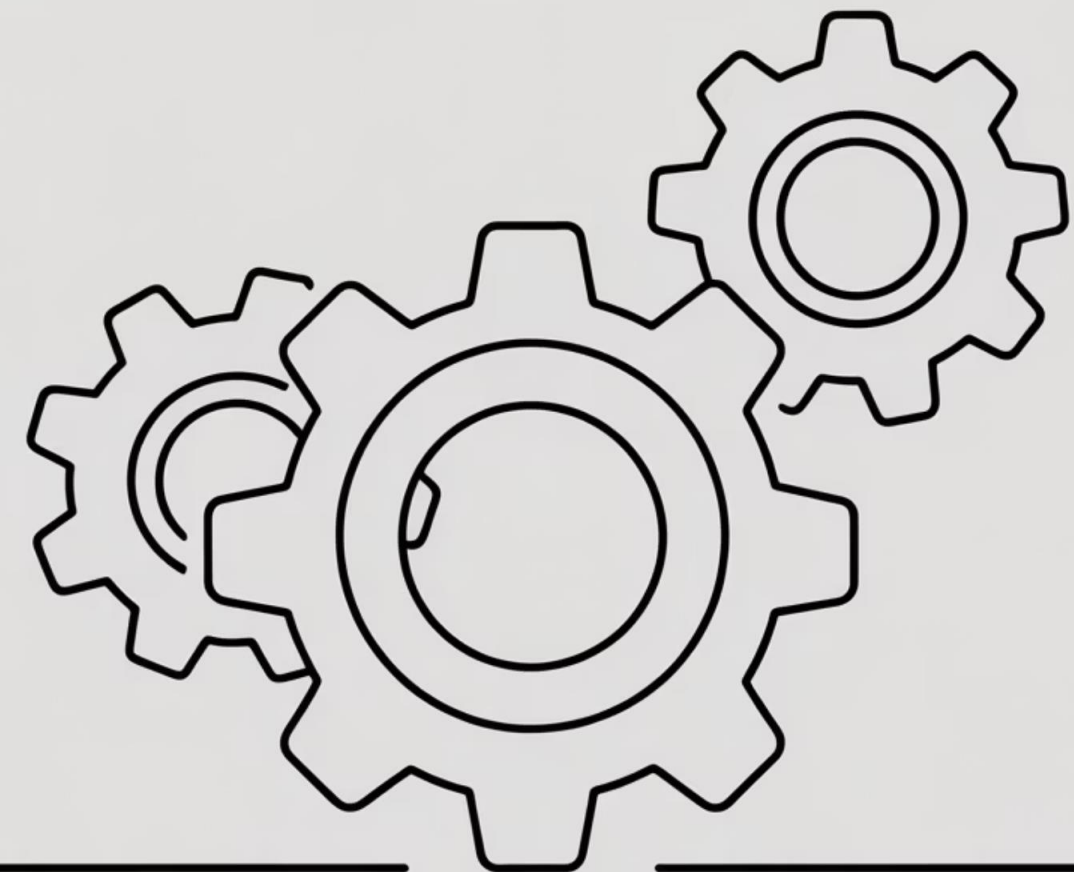
Анализ трансформации подходов к разработке программного обеспечения - от классического программирования к самообучающимся системам



Три эпохи программного обеспечения

Концепция трех эпох программного обеспечения, сформулированная **Андреем Карпати**, представляет революционную эволюцию от традиционного программирования к современным AI-системам. Каждая эпоха характеризуется уникальными технологическими прорывами, выдающимися персоналиями и фундаментальными изменениями в подходе к созданию программ.

Эти эпохи не просто сменяют друг друга — они сосуществуют, взаимодополняют и трансформируют всю индустрию разработки программного обеспечения.





How I use LLMs

1,9 млн просмотров • 6 месяцев назад



Deep Dive into LLMs like ChatGPT

3,5 млн просмотров • 7 месяцев назад



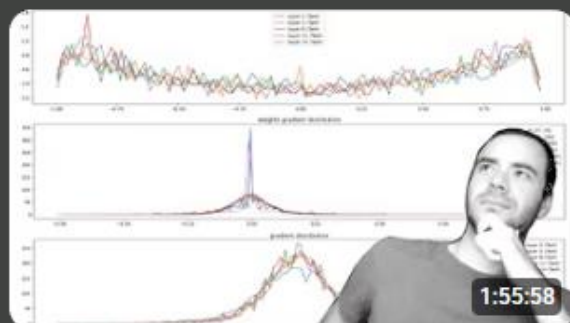
[1hr Talk] Intro to Large Language Models

3 млн просмотров • 1 год назад

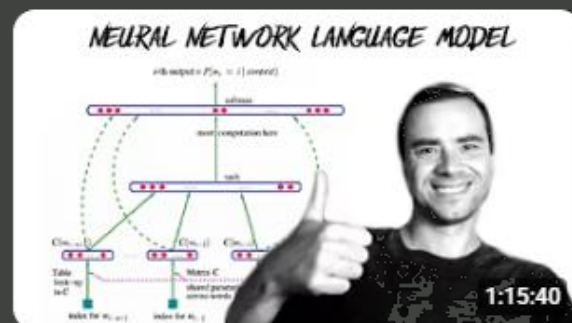


Let's build GPT: from scratch, in code, spelled out.

6,3 млн просмотров • 2 года назад



Building makemore Part 3: Activations & Gradients, BatchNorm



Building makemore Part 2: MLP

461 тыс. просмотров • 3 года назад

Андрей Карпаты й: Пионер ИИ и создатель концепций Software 2.0 и vibe-coding

Словацко-канадский специалист в области информатики и одна из самых влиятельных фигур в развитии современного искусственного интеллекта. От OpenAI до Tesla — путь визионера, который изменил представление о программировании.

Путь от Братиславы до вершин ИИ

Биография

Андрей Карпатый (род. 23 октября 1986) родился в Братиславе, переехал в Торонто в 15 лет. Получил степени в Университете Торонто и УБК, защитил докторскую в Стэнфорде под руководством Фей-Фей Ли.



Наследие визионера: От образования к трансформации индустрии

Образовательный вклад

Карпатый известен как ведущий популяризатор ИИ-образования, вдохновленный Ричардом Фейнманом в стремлении сделать сложное простым и понятным.

- **CS 231n** — курс в Стэнфорде вырос с 150 до 750 студентов
- **nanoGPT** — простейший репозиторий для обучения GPT
- **micrograd** — минималистичный движок дифференцирования
- Серия "Zero to Hero" на YouTube (800K+ просмотров)

Влияние на индустрию

Концепции Карпатога — не просто академические идеи, а реальная трансформация разработки ПО. Его идеи о Software 2.0 уже применяются в Tesla Autopilot, системах компьютерного зрения и обработки естественного языка по всему миру.

35

новаторов MIT

Включен в престижный список до 35 лет в 2020 году

100

влиятельных Time

Признан одним из 100 самых влиятельных людей в ИИ в 2024

800K+

просмотров

Его образовательные видео набрали огромную аудиторию

Терминология от лукавого

 Pinned

 **Andrej Karpathy**  @karpathy · Jan 24, 2023

[Promote](#)  ...

The hottest new programming language is English

 1.1K  7K  44K  7.4M  

 **Andrej Karpathy**  @karpathy

[Promote](#)  ...

There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

Vibe coding

 24 languages 

Article [Talk](#)

[Read](#) [Edit](#) [View history](#) [Tools](#) 

From Wikipedia, the free encyclopedia



This article **may contain an excessive number of citations**. Please help [remove low-quality or irrelevant citations](#). (June 2025) ([Learn how and when to remove this message](#))

Vibe coding (or **vibecoding**) is an approach to producing software by using [artificial intelligence](#) (AI), where a person describes a problem in a few [natural language](#) sentences as a [prompt](#) to a [large language model](#) (LLM) tuned for coding. The LLM generates [software](#) based on the description, shifting the programmer's role from manual coding to guiding, testing, and refining the AI-generated [source code](#).^{[1][2][3]}

Advocates of vibe coding say that it allows even [amateur programmers](#) to produce software without the extensive training and skills required for [software engineering](#).^[4] The term was introduced by [Andrej Karpathy](#) in February 2025^{[5][2][4][1]} and listed in the [Merriam-Webster Dictionary](#) the following month as a "slang & trending" noun.^[6]

План презентации

1

Определения парадигм

Ключевые принципы и эволюция от Software 1.0 к 3.0

2

Сравнительный анализ

Преимущества, недостатки и применимость каждого подхода

3

Критический разбор

Почему парадигмы дополняют, а не заменяют друг друга

4

Анализ кейсов

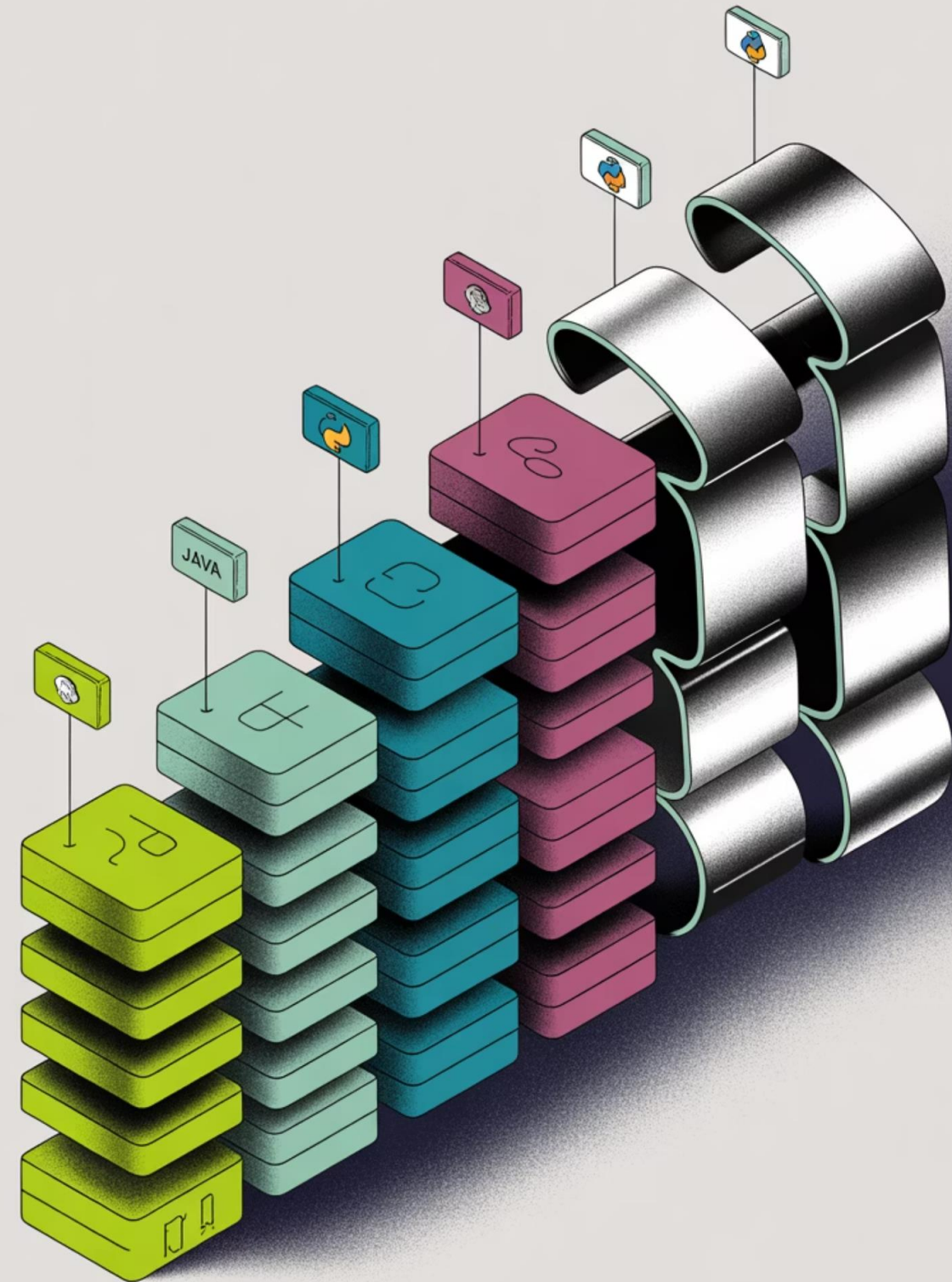
Примеры реализации каждой парадигмы в современных системах

5

Рекомендации

Необходимые компетенции и образовательная траектория для специалистов

Часть 1: Определения парадигм



Software 1.0

Эра традиционного программирования



Истоки программирования

1843 год: Первый алгоритм

Ада Лавлейс создает первый алгоритм для аналитической машины Чарльза Бэббиджа, заложив фундаментальные основы программирования. Её работа содержала не только инструкции для вычислений, но и понимание потенциала машин для обработки символов и создания музыки.

Diagram for the computation by the Engine of the Numbers of Bernoulli. See Note G. (page 722 et seq.)

Number of Operations.	Nature of Operations.	Variables acted upon.	Variables receiving results.	Indication of change in the value on any Variable.	Statement of Results.	Data.										Working Variables.				Result Variables.			
						v_1	v_2	v_3	ov_4	ov_5	ov_6	ov_7	ov_8	ov_9	ov_{10}	ov_{11}	ov_{12}	ov_{13}	ov_{14}	v_{21}	v_{22}	v_{23}	ov_{24}
						1	2	4	0	0	0	0	0	0	0	0	0	0	0	B_1 in fraction.	B_2 in fraction.	B_3 in fraction.	B_7 in fraction.
						1	2	4	0	0	0	0	0	0	0	0	0	0	0	B_1	B_2	B_3	B_7
1	$\times$	$v_2 \times v_3$	v_4, v_6, v_9	$v_2 = v_2$ $v_4 = v_4$ $v_6 = v_6$ $v_9 = v_9$	$= 2n$	...	2	n	2n	2n	2n												
2	$-$	$v_4 - v_1$	$2v_4$	$v_4 = v_4$ $v_1 = v_1$	$= 2n - 1$	...	1	...	2n - 1														
3	$+$	$v_5 + v_1$	$2v_5$	$v_5 = v_5$ $v_1 = v_1$	$= 2n + 1$	...	1	...	2n + 1														
4	$+$	$v_5 + v_4$	v_{11}	$v_5 = v_5$ $v_4 = v_4$	$= 2n + 1$	...	...	...	0	0													
5	$+$	$v_{11} + v_2$	$2v_{11}$	$v_{11} = v_{11}$ $v_2 = v_2$	$= \frac{1}{2} \cdot \frac{2n-1}{2n+1}$	...	2	...															
6	$-$	$v_{12} - v_{11}$	v_{12}	$v_{12} = v_{12}$ $v_{11} = v_{11}$	$= \frac{1}{2} \cdot \frac{2n-1}{2n+1} = A_0$	...	...	...															
7	$-$	$v_8 - v_1$	v_{10}	$v_8 = v_8$ $v_1 = v_1$	$= n - 1 (= 3)$	...	1	n															
8	$+$	$v_2 + v_7$	v_7	$v_2 = v_2$ $v_7 = v_7$	$= 2 + 0 = 2$	...	2	...															
9	$+$	$v_6 + v_2$	$2v_{11}$	$v_6 = v_6$ $v_2 = v_2$	$= \frac{2n}{2} = A_1$	...	...	...			2n	2											
10	$\times$	$v_{21} \times v_{11}$	v_{12}	$v_{21} = v_{21}$ $v_{11} = v_{11}$	$= B_1 \cdot \frac{2n}{2} = B_1 A_1$	...	...	...															
11	$+$	$v_{12} + v_{11}$	$2v_{12}$	$v_{12} = v_{12}$ $v_{11} = v_{11}$	$= \frac{1}{2} \cdot \frac{2n-1}{2n+1} + B_1 \cdot \frac{2n}{2}$	...	...	...															
12	$-$	$v_{10} - v_1$	$2v_{10}$	$v_{10} = v_{10}$ $v_1 = v_1$	$= n - 2 (= 2)$	...	1	...															
13	$-$	$v_6 - v_1$	v_6	$v_6 = v_6$ $v_1 = v_1$	$= 2n - 1$	...	1	...			2n - 1												
14	$+$	$v_1 + v_2$	$2v_2$	$v_1 = v_1$ $v_2 = v_2$	$= 2 + 1 = 3$	...	1	...				3											
15	$+$	$v_6 + v_2$	v_6	$v_6 = v_6$ $v_2 = v_2$	$= \frac{2n-1}{2}$	...	...	...			2n - 1	3											
16	$\times$	$v_6 \times v_{11}$	v_{11}	$v_6 = v_6$ $v_{11} = v_{11}$	$= \frac{2n-1}{2} \cdot \frac{2n-1}{2}$	...	...	...				0											
17	$-$	$v_6 - v_1$	v_6	$v_6 = v_6$ $v_1 = v_1$	$= 2n - 2$	...	1	...			2n - 2												
18	$+$	$v_1 + v_2$	$2v_2$	$v_1 = v_1$ $v_2 = v_2$	$= 3 + 1 = 4$	...	1	...				4											
19	$+$	$v_6 + v_2$	v_6	$v_6 = v_6$ $v_2 = v_2$	$= \frac{2n-1}{2} \cdot \frac{2n-1}{2} = A_3$	...	...	...			2n - 2	4											
20	$\times$	$v_6 \times v_{11}$	v_{11}	$v_6 = v_6$ $v_{11} = v_{11}$	$= \frac{2n-1}{2} \cdot \frac{2n-1}{2} = A_3$	...	...	...				0											
21	$\times$	$v_{21} \times v_{11}$	v_{12}	$v_{21} = v_{21}$ $v_{11} = v_{11}$	$= B_1 \cdot \frac{2n-1}{2} \cdot \frac{2n-1}{2} = B_1 A_3$	...	...	...															
22	$+$	$v_{12} + v_{11}$	$2v_{12}$	$v_{12} = v_{12}$ $v_{11} = v_{11}$	$= A_0 + B_1 A_1 + B_2 A_3$	...	...	...															
23	$-$	$v_{10} - v_1$	$2v_{10}$	$v_{10} = v_{10}$ $v_1 = v_1$	$= n - 3 (= 1)$	...	1	...															
Here follows a repetition of Operations thirteen to twenty-three.																							
24	$+$	$v_{12} + v_{11}$	$2v_{12}$	$v_{12} = v_{12}$ $v_{11} = v_{11}$	$= B_7$	...	...	...															
25	$+$	$v_1 + v_2$	$2v_2$	$v_1 = v_1$ $v_2 = v_2$	$= n + 1 = 4 + 1 = 5$	...	1	n + 1			0	0											



<https://ru.wikipedia.org/wiki/%D0%9B%D0%B0%D0%B2%D0%BB%D0%B5%D0%B9%D1%81,%D0%90%D0%B4%D0%B0>

https://en.wikipedia.org/wiki/Ada_Lovelace#/media/File:Diagram_for_the_computation_of_Bernoulli_numbers.jpg

Рождение языков программирования

1944-1945: Plankalkül

Конрад Цузе разрабатывает первый настоящий язык программирования высокого уровня. Язык включал условные операторы, циклы и работу с массивами - концепции, которые остаются актуальными по сей день.

1972: Язык C

Деннис Ричи в Bell Labs разрабатывает язык C, который становится основой для множества современных языков и операционных систем, включая Linux и Windows.

1

2

1957: FORTRAN

Команда Джона Бэкуса в IBM создает первый коммерчески доступный язык высокого уровня, революционизирующий научные вычисления и открывающий путь к массовому программированию.

3

Ключевые принципы Software 1.0

Детерминированная логика

Каждое выполнение программы при одинаковых входных данных гарантированно дает идентичный результат. Поведение системы полностью предсказуемо и воспроизводимо.

Полный контроль

Разработчик имеет абсолютный контроль над всеми аспектами программы — от управления памятью до логики принятия решений.

Хрупкость систем

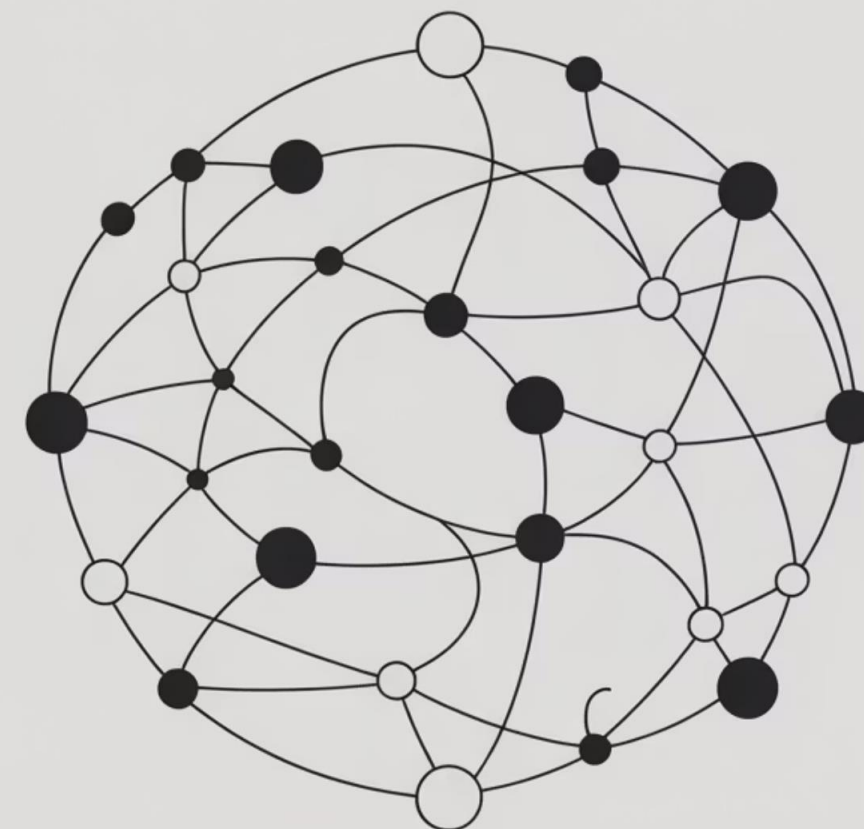
Программы плохо справляются с неопределенностью и неожиданными ситуациями, требуя предварительного программирования всех возможных сценариев.

Ограничения в распознавании

Традиционные алгоритмы неэффективны в задачах распознавания образов, обработки естественного языка и других "интуитивных" задач.

Software 2.0

Эра нейронных сетей



Фундаментальные основы нейронных сетей

1943: Модель нейронов

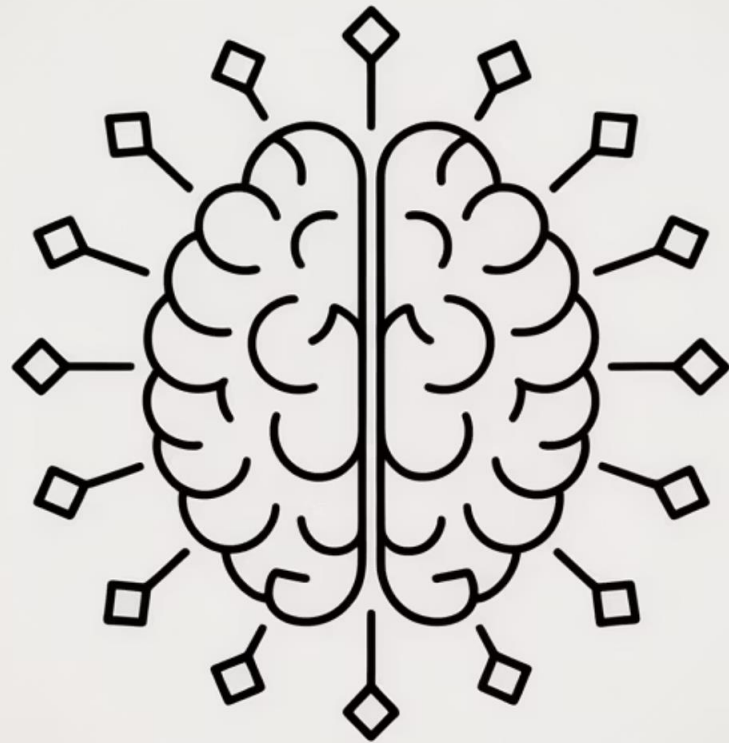
Уоррен МакКаллок и Уолтер Питтс предлагают первую математическую модель искусственных нейронов, основанную на биологических принципах работы мозга. Их работа заложила теоретические основы для всех будущих нейронных сетей.

1969: Первая "зима ИИ"

Марвин Мински и Сеймур Паперт публикуют критический анализ ограничений простых нейросетей, что приводит к значительному снижению финансирования исследований ИИ на два десятилетия.

1958: Перцептрон

Фрэнк Розенблатт создает перцептрон - первую обучающуюся нейронную сеть, способную автоматически настраивать свои параметры на основе примеров. Это был революционный шаг от программирования к обучению.



Возрождение нейронных сетей

1986 год стал поворотным моментом в истории искусственного интеллекта. Повторное открытие и популяризация алгоритма обратного распространения ошибки (backpropagation) позволило эффективно обучать многослойные нейронные сети.

Этот алгоритм решил фундаментальную проблему: как обучать скрытые слои в глубоких нейронных сетях. Впервые стало возможно создавать сложные нелинейные модели, способные решать задачи, недоступные простым перцептронам.

Синергия трех компонентов в развитии нейронных сетей

История развития искусственного интеллекта демонстрирует, что **прогресс невозможен без синхронного развития** трех фундаментальных компонентов:



Точка перелома: AlexNet (2012)

AlexNet, разработанный **Алексом Крижевским**, **Ильей Суцкевером** и **Джефффри Хинтоном**, снизил ошибку классификации изображений с 26% до 15,3%, совершив революцию в компьютерном зрении.

Три кита успеха AlexNet:



Архитектура CNN

Автоматическое извлечение признаков без ручной разработки, глубина в 8 слоёв, функция активации ReLU



Использование GPU

Обучение на двух NVIDIA GTX 580 с использованием CUDA параллельно ускорило вычисления в десятки раз



ImageNet

Датасет из 1,2 миллиона изображений в 1000 категориях обеспечил достаточно данных для обучения глубокой сети

“Ilya thought we should do it, Alex made it work, and I got the Nobel Prize.”

Джефффри Хинтон

Software 2.0: Революция в программировании

В 2017 году Карпатый представил концепцию **Software 2.0** - фундаментальный сдвиг в парадигме разработки программного обеспечения, который изменил понимание того, что такое программирование.

Software 1.0

Традиционное программирование на Python, C++, где программист пишет явные инструкции для компьютера, определяя каждую строку кода

Software 2.0

Код, написанный на "языке" весов нейронной сети. Миллионы параметров, которые человек не может написать напрямую

Ключевые принципы новой парадигмы

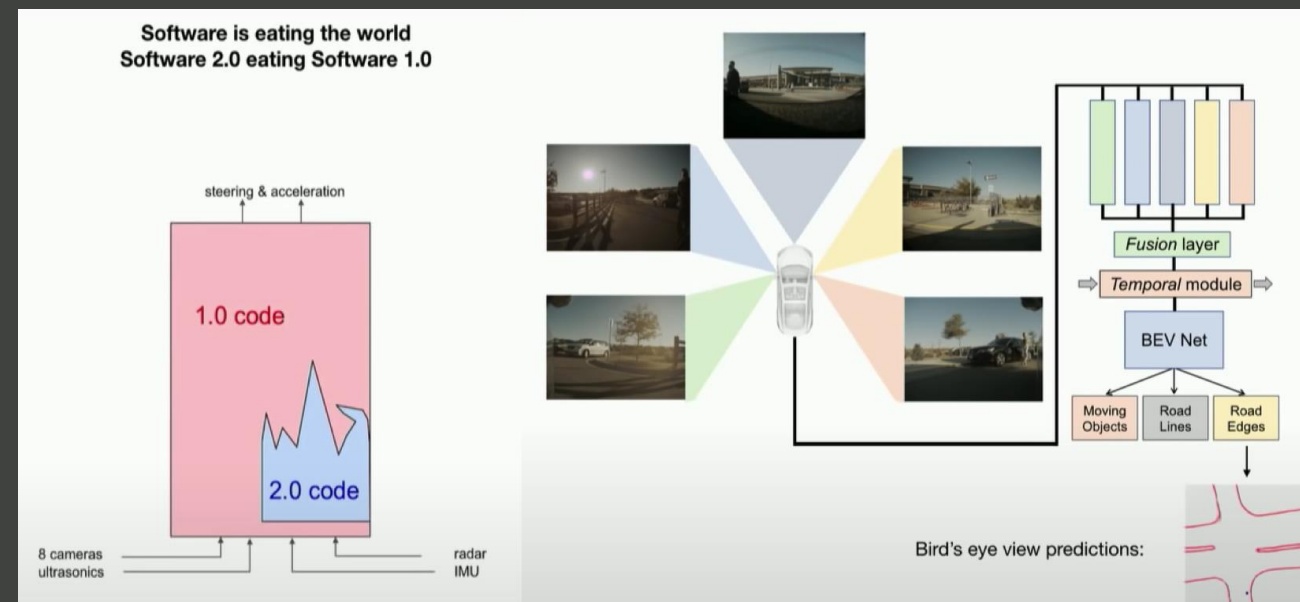
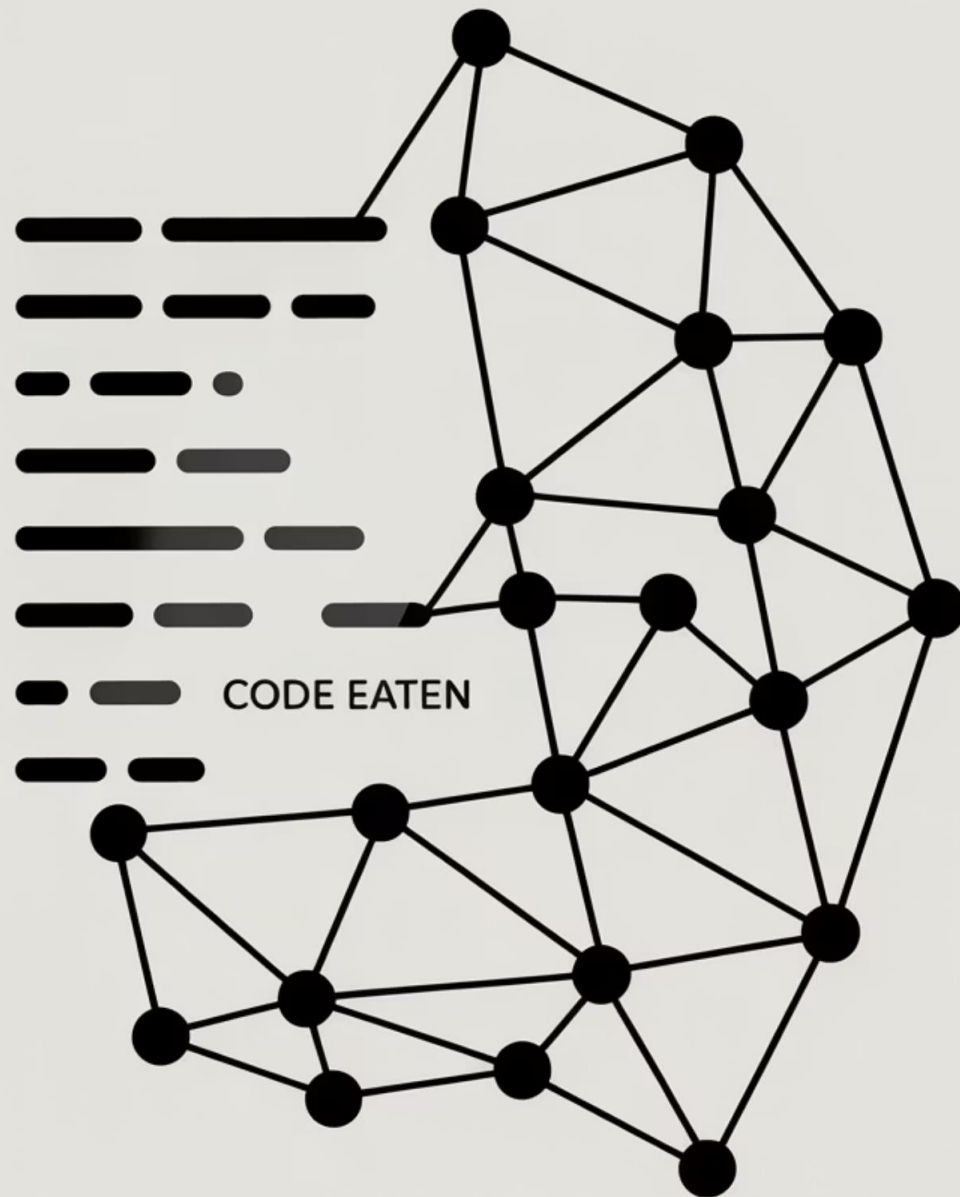
- Разработчик определяет цель и архитектуру, а не детали реализации
- "Компиляция" происходит через обучение на данных
- Основная работа смещается к курированию и разметке данных
- Программа "эволюционирует" через градиентный спуск

Практическое применение этой концепции в Tesla Autopilot показало её эффективность: постепенный переход от традиционного C++ к нейронным сетям для обработки данных с камер в реальном времени.

Принцип "Software 2.0 съедает Software 1.0« (картинка сворована у Корпаты)

Карпати наблюдал удивительный феномен: нейронные сети не просто дополняли традиционный код, а активно заменяли его. Каждое обновление системы означало меньше рукописных алгоритмов и больше обученных весов.

Этот процесс был необратимым — как только нейронная сеть начинала превосходить традиционный алгоритм по производительности, возврат к старому подходу становился невозможным. Данные и обучение побеждали человеческую интуицию и явное программирование.



Ключевые принципы Software 2.0



Веса как код

"Код" теперь состоит из миллионов весов нейронной сети, обученных на данных, а не из инструкций, написанных программистами.



Программирование данными

Вместо написания явных правил разработчики собирают и размечают данные, позволяя алгоритмам самостоятельно выявлять закономерности.



Автоматическая оптимизация

Алгоритмы обучения, такие как градиентный спуск, автоматически оптимизируют производительность системы без вмешательства человека.



Превосходство в распознавании

Нейронные сети демонстрируют беспрецедентную производительность в задачах распознавания образов, превосходя человеческие способности в многих областях.



Software 3.0

Эра больших языковых моделей

Трансформеры и современность (2017-настоящее время)

"Attention is All You Need" (2017)

Исследователи Google Brain представили архитектуру трансформеров, использующую **механизм внимания (attention)** вместо рекуррентных связей, что привело к:

- Возможности **параллельных вычислений** (в отличие от RNN/LSTM)
- Способности учитывать **дальние зависимости** в последовательностях
- Лучшему **масштабированию** на больших объемах данных



Эволюция GPT: от эксперимента к революции

Июнь 2018: GPT-1

1

OpenAI представляет первую модель серии с 117 миллионами параметров. Модель демонстрирует способность к unsupervised предобучению на текстах и последующей тонкой настройке на конкретных задачах.

Июнь 2020: GPT-3

3

Модель с 175 миллиардами параметров стала качественным скачком, демонстрируя способности few-shot обучения - решения новых задач без дополнительного обучения, только на основе примеров в промпте.

2

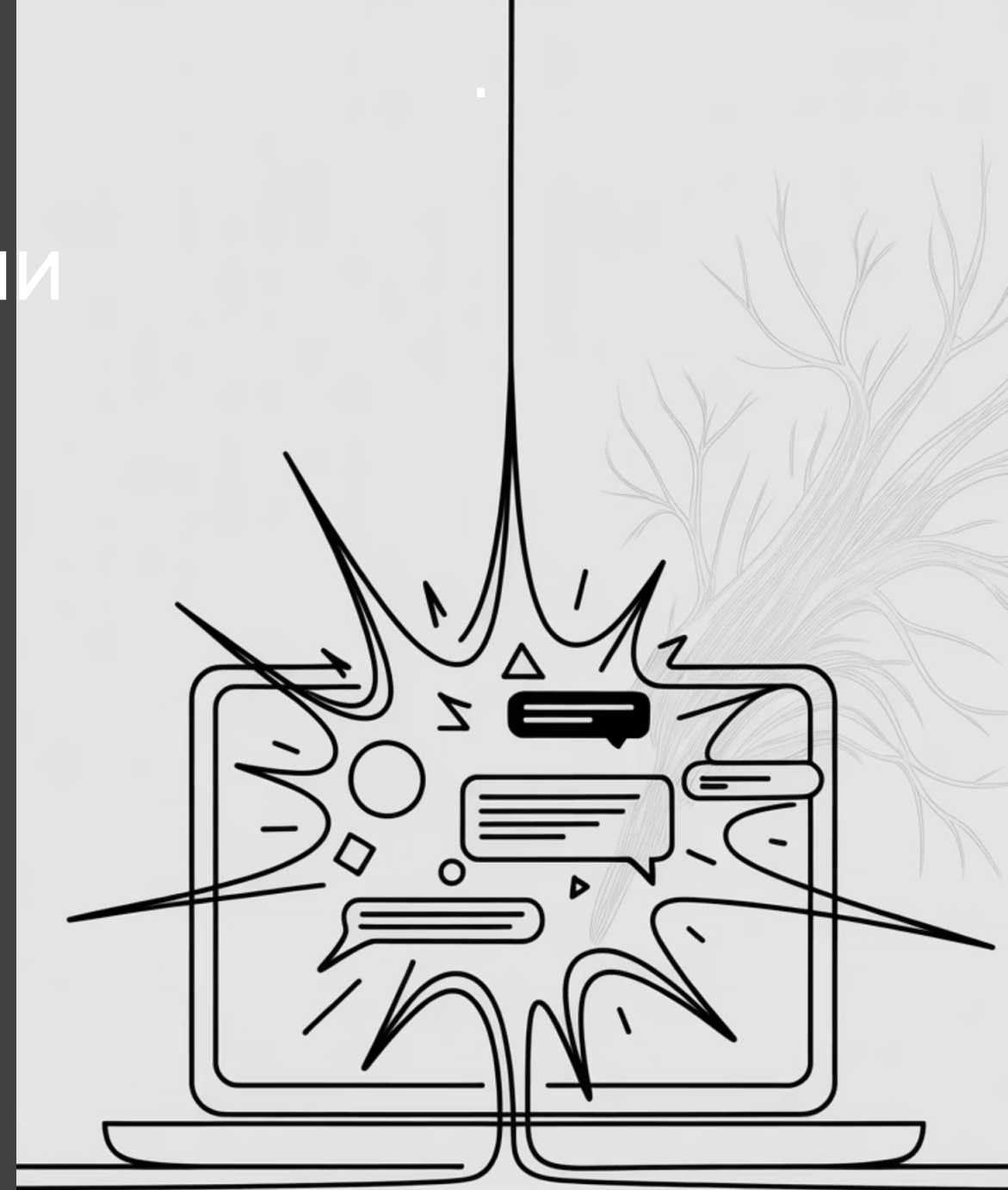
Февраль 2019: GPT-2

Модель с 1.5 миллиардами параметров изначально не была полностью опубликована из-за опасений потенциального злоупотребления. GPT-2 продемонстрировала впечатляющую способность генерировать связные тексты.

ChatGPT: Поворотный момент истории

30 ноября 2022 года стало датой, которая разделила историю искусственного интеллекта на "до" и "после". Запуск ChatGPT на основе GPT-3.5 впервые продемонстрировал широкой публике практическую применимость больших языковых моделей.

В отличие от предыдущих AI-систем, ChatGPT был интуитивно понятен обычным пользователям. Не нужно было изучать специальные языки программирования или сложные интерфейсы — достаточно было говорить на естественном языке.



Взрывной рост ChatGPT

100M

Пользователей за 2 месяца

К январю 2023 года ChatGPT достиг 100 миллионов активных пользователей, став самым быстрорастущим потребительским приложением в истории.

180.5M

Пользователей в 2024

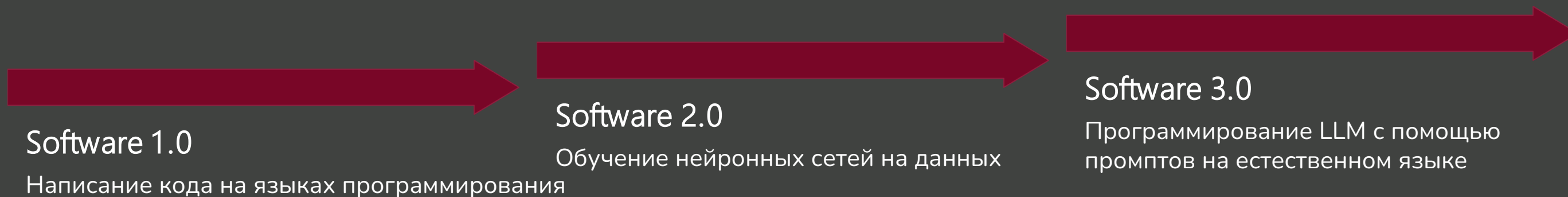
К середине 2024 года аудитория выросла до 180.5 миллионов пользователей, подтверждая устойчивый интерес к технологии.

2

Месяца до 100M

Предыдущие рекордсмены: Instagram (2.5 года), TikTok (9 месяцев). ChatGPT побил все рекорды скорости роста.

Software 3.0 и vibe-coding: Будущее уже здесь



Vibe-coding: Революционный подход

В феврале 2025 года Карпатый ввел термин "**vibe-coding**" - методологию разработки, где программист полностью полагается на ИИ-ассистентов, описывая задачи на естественном языке.

"Полностью отдайся вайбам, прими экспоненциальный рост возможностей и забудь, что код вообще существует"
Андрей Карпатый

Философия подхода

Полное доверие к "вайбам" и отказ от детального понимания кода. ИИ генерирует весь код, разработчик тестирует и дает обратную связь.

Практическое применение

Идеально подходит для быстрого создания прототипов и "выходных проектов", когда важна скорость, а не глубокое понимание.

Принципы Software 3.0



Естественный язык как интерфейс

Программирование через промпты на обычном человеческом языке заменяет традиционные языки программирования. Барьер входа снижается до умения формулировать задачи словами.



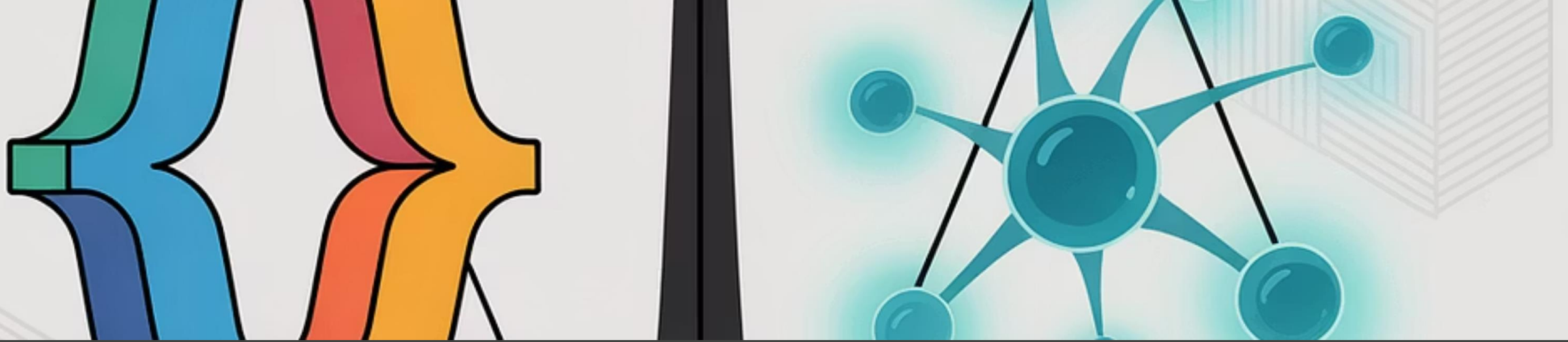
LLM как компьютеры

Большие языковые модели функционируют как программируемые системы, способные интерпретировать инструкции, хранить контекст и выполнять сложные многоэтапные задачи.



Поглощение предыдущих эпох

"Software 3.0 поглощает Software 1.0 и 2.0" — новая парадигма не только дополняет, но активно заменяет традиционные подходы к программированию.



Часть 2: Сравнительный анализ

Пример (сворованный у Корпаты): Анализ тональности текстов

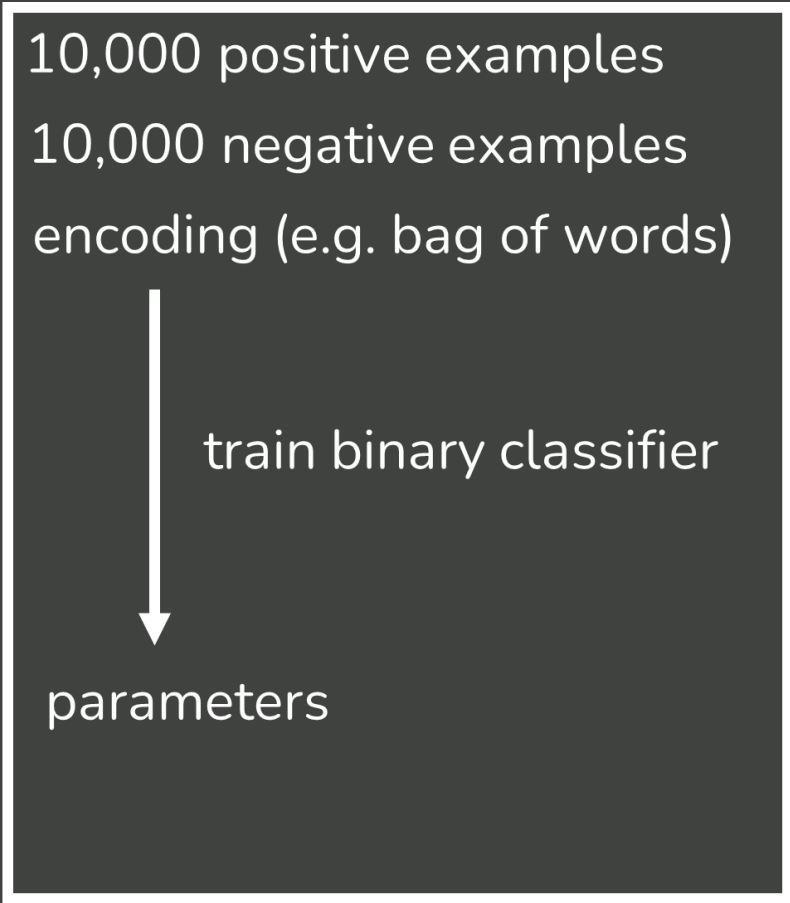
Software 1.0

```
python
def simple_sentiment(review: str) -> str:
    """Return 'positive' or 'negative' based on a tiny keyword lexicon."""
    positive = {
        "good", "great", "excellent", "amazing", "wonderful", "fantastic",
        "awesome", "loved", "love", "like", "enjoyed", "superb", "delightful"
    }
    negative = {
        "bad", "terrible", "awful", "poor", "boring", "hate", "hated",
        "dislike", "worst", "dull", "disappointing", "mediocre"
    }

    score = 0
    for word in review.lower().split():
        w = word.strip(".,!?:;")
        if w in positive:
            score += 1
        elif w in negative:
            score -= 1

    return "positive" if score >= 0 else "negative"
```

Software 2.0



Software 3.0

You are a sentiment classifier. For every review that appears between the tags
<REVIEW> ... </REVIEW>, respond with **exactly one word**, either
POSITIVE or NEGATIVE (all-caps, no punctuation, no extra text).

Example 1
<REVIEW>I absolutely loved this film—the characters were engaging and
the ending was perfect.</REVIEW>
POSITIVE

Example 2
<REVIEW>The plot was incoherent and the acting felt forced; I regret
watching it.</REVIEW>
NEGATIVE

Example 3
<REVIEW>An energetic soundtrack and solid visuals almost save it, but
the story drags and the jokes fall flat.</REVIEW>
NEGATIVE

Now classify the next review.

Преимущества каждой парадигмы

Software 1.0

- Прозрачность работы алгоритмов
- Предсказуемость поведения
- Детерминированные результаты
- Точный контроль исполнения

Software 2.0

- Гибкость в решении сложных задач
- Способность работать с неструктурированными данными
- Адаптивность к неявным паттернам
- Обобщение на новые примеры

Software 3.0

- Масштабируемость функциональности
- Ускоренная разработка новых возможностей
- Самооптимизация в продакшене
- Адаптация к изменению условий без перепрограммирования

Недостатки и риски

Software 1.0

- Долгий цикл внесения изменений
- Сложность масштабирования при росте функциональности
- Ограниченная способность к адаптации
- Высокая стоимость обслуживания по мере роста кодовой базы

Software 2.0

- Зависимость от качества обучающих данных
- Непрозрачность принятия решений ("чёрный ящик")
- Сложность интерпретации результатов
- Высокие требования к вычислительным ресурсам

Software 3.0

- Риски непредсказуемого поведения
- Чрезвычайно высокая сложность архитектуры
- Потребность в постоянном мониторинге
- Сложность отладки при возникновении проблем

ОСНОВНОЙ ВЫВОД

Парадигмы Software 1.0, 2.0 и 3.0 развиваются эволюционно, а не вытесняют друг друга

Каждая из них вносит новые возможности: от жёстко запрограммированных алгоритмов (1.0) через обучение на данных (2.0) к автономным, контекстно-адаптивным системам (3.0).

Переход на Software 3.0 меняет баланс затрат, увеличивая инвестиции в инфраструктуру данных и ML-платформы, но даёт выигрыш в масштабируемости и скорости развития функциональности при условии адекватного human-in-the-loop контроля.

Часть 3: Критический разбор

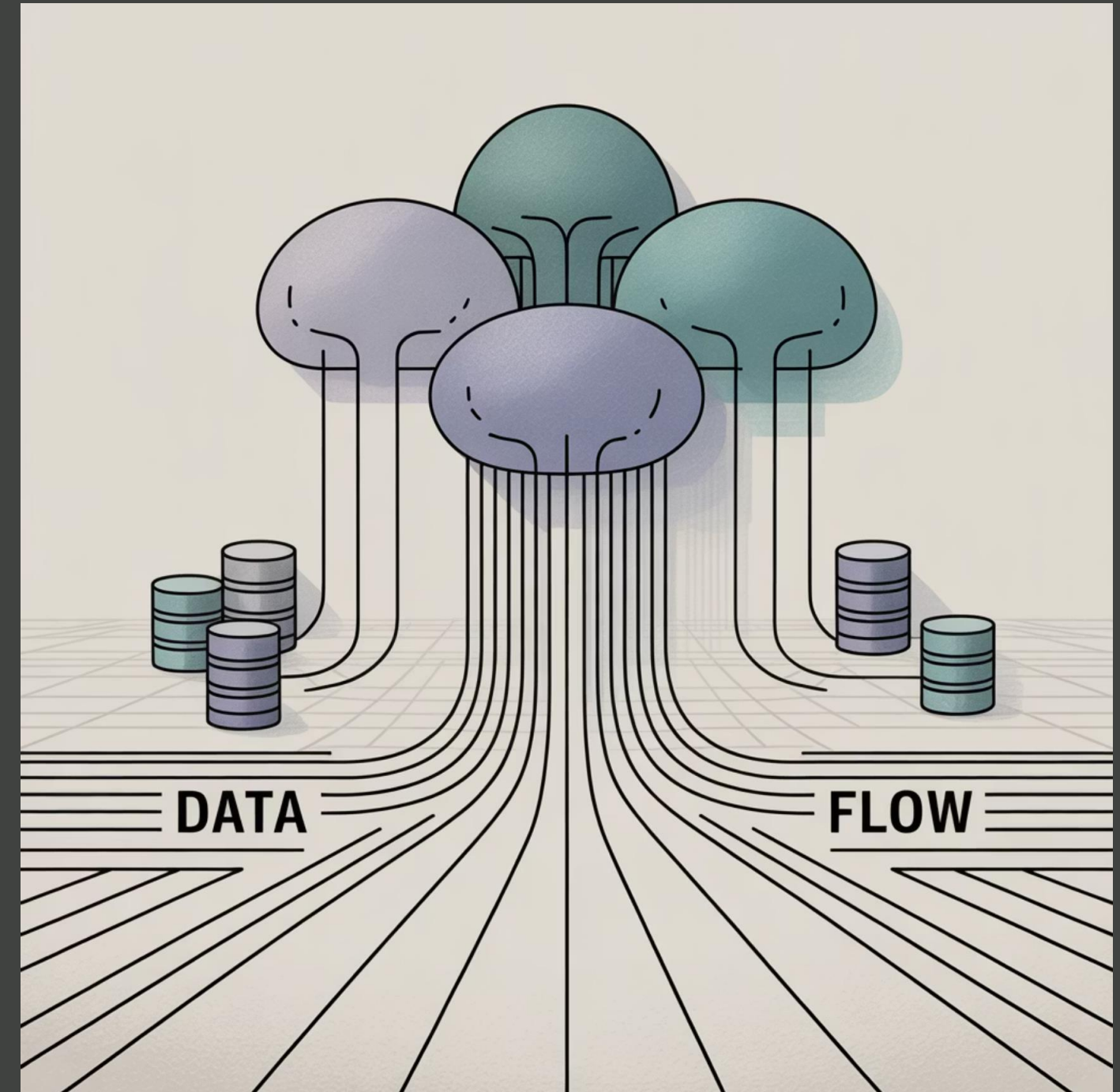


Software Harmony

Почему Software 2.0 дополнил, а не вытеснил 1.0

Классические алгоритмы (1.0) по-прежнему лежат в основе современных ML-систем (2.0):

- Предобработка и аннотирование данных выполняются скриптами 1.0
- Инфраструктура обучения (пакеты трансформации, загрузки батчей) код 1.0
- ML-модели "питаются" данными, подготовленными детерминированными алгоритмами



ML-модели встраиваются в экосистему традиционного кода, а не заменяют её полностью

Иллюзия автономности Software 3.0

Без надлежащего мониторинга и человеческого контроля "чёрные ящики" автономных систем могут:

Выдавать некорректные решения

При столкновении с данными, отличными от обучающей выборки

Усиливать смещения

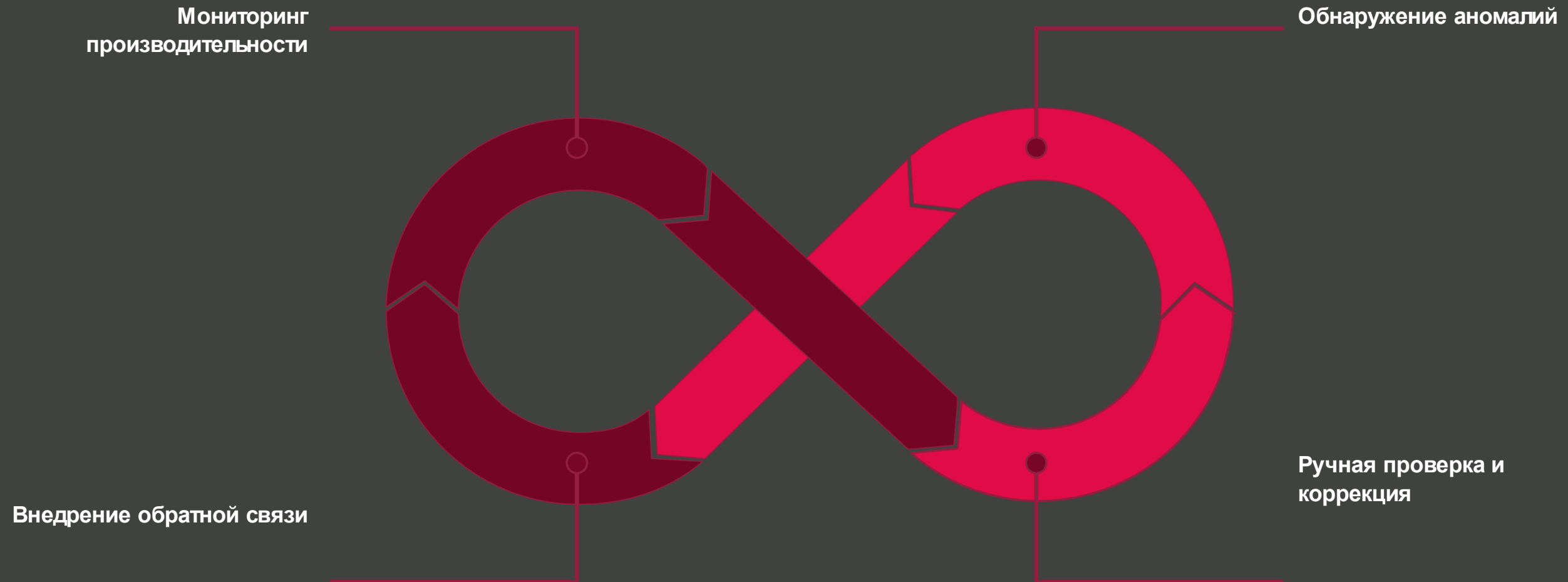
Системы могут "самоусиливать" искажения, присутствующие в исходных данных

Дегradировать без заметных признаков

Ухудшение качества может происходить постепенно и незаметно



Необходимость human-in-the-loop



Для эффективной работы Software 3.0 необходим контур с участием экспертов, которые:

- Проверяют аномалии в решениях системы
- Корректируют модели при обнаружении проблем
- Определяют границы применимости автономных решений
- Оценивают этические последствия принимаемых решений

Экономический аспект трансформации

Переход на Software 3.0 меняет структуру затрат и модели монетизации:

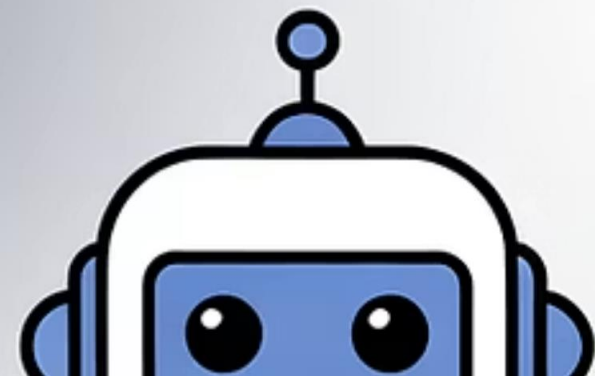
Структура затрат

- ↑ **Увеличивается CapEx** на серверы, GPU-кластеры и платформы MLOps
- ↓ **Снижаются OpEx** за счёт автоматизации рутинных обновлений и feature-релизов

Модели монетизации

- SaaS-модели переходят к **pay-per-use ML-API** вместо лицензий ПО
- Появляются **подписки на данные** и непрерывное обучение (data-as-a-service)

По мнению А. Karpathy, Software 3.0 "eats" 1.0/2.0, так как интеллектуальные компоненты генерируются и адаптируются автоматически, снижая долю ручной разработки и поддержки.



Часть 4: Анализ кейсов

Примеры Software 1.0



Библиотека STL (C++)

Стандартная библиотека шаблонов с детерминированными алгоритмами и структурами данных



Apache HTTP Server

Классический веб-сервер с чётко прописанной логикой обработки HTTP-запросов



OpenSSL

Криптографическая библиотека с алгоритмами шифрования, реализованными "вручную"

Эти системы характеризуются **прозрачностью работы, предсказуемостью поведения и относительной простотой верификации**. Их разработка требует глубокого понимания алгоритмов и языков программирования.

Как Антон Юрьевич изучал PowerShell

Имя	Дата изменения	Тип	Размер
 p (5).pdf	01.09.2025 14:13	Документ Adobe ...	1 693 КБ
 p (4).pdf	01.09.2025 14:13	Документ Adobe ...	1 693 КБ
 p (3).pdf	01.09.2025 14:13	Документ Adobe ...	1 693 КБ
 p (2).pdf	01.09.2025 14:13	Документ Adobe ...	1 693 КБ
 p (1).pdf	01.09.2025 14:13	Документ Adobe ...	1 693 КБ

```
Get-ChildItem *.pdf | Rename-Item -NewName { $_.Name -replace ' ', '' }
```

```
Get-ChildItem *.pdf | Rename-Item -NewName { $_.Name -replace '\(', '' }
```

```
Get-ChildItem *.pdf | Rename-Item -NewName { $_.Name -replace '\)', '' }
```

https://gitverse.ru/dayekb/moodle_to_googlesheets

Примеры Software 2.0

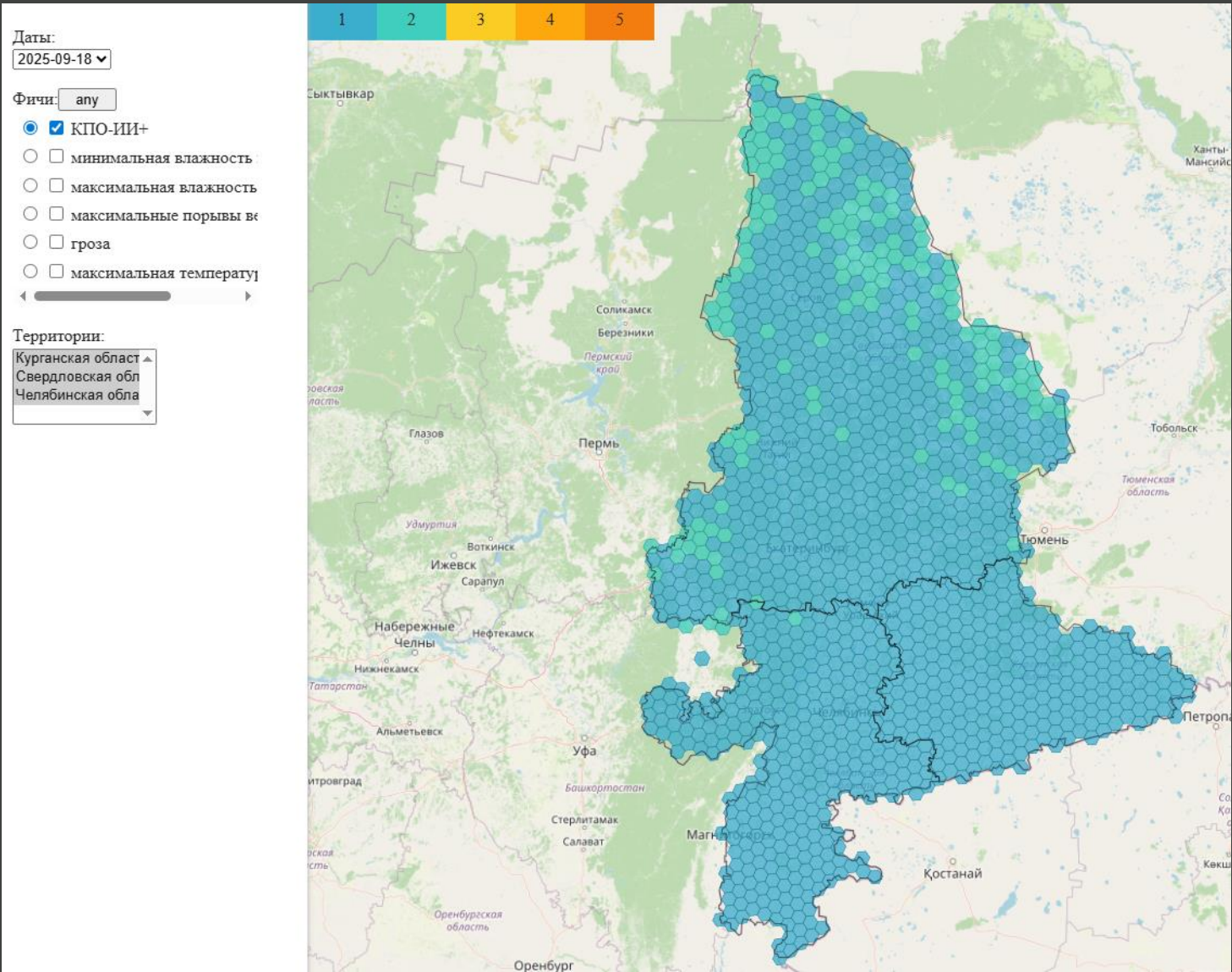


Фреймворки и инструменты ML:

- TensorFlow
- PyTorch
- Hugging Face Transformers
- OpenAI Codex
- Scikit-learn

Эти системы позволяют **создавать и обучать модели**, которые "программируются" через данные, а не через явный код. Качество работы этих систем напрямую зависит от **качества и объёма данных**, используемых для обучения.

Как Антон Юрьевич пытался пожары предсказывать



https://gitverse.ru/laboratoriya_ii_sber_urfu/fire_hazard

Примеры Software 3.0



AutoML-системы

DataRobot и аналогичные платформы, автоматически выбирающие архитектуры моделей и гиперпараметры



GPT-платформы

Большие языковые модели, способные к контекстной адаптации и многозадачности



RL-системы

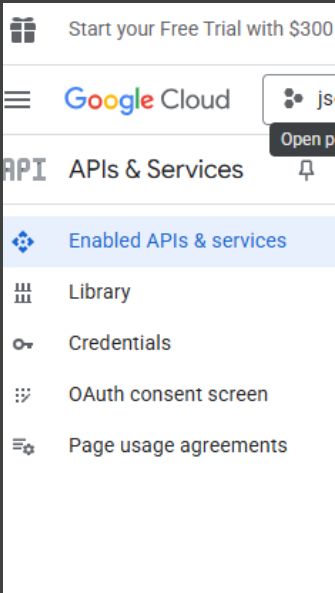
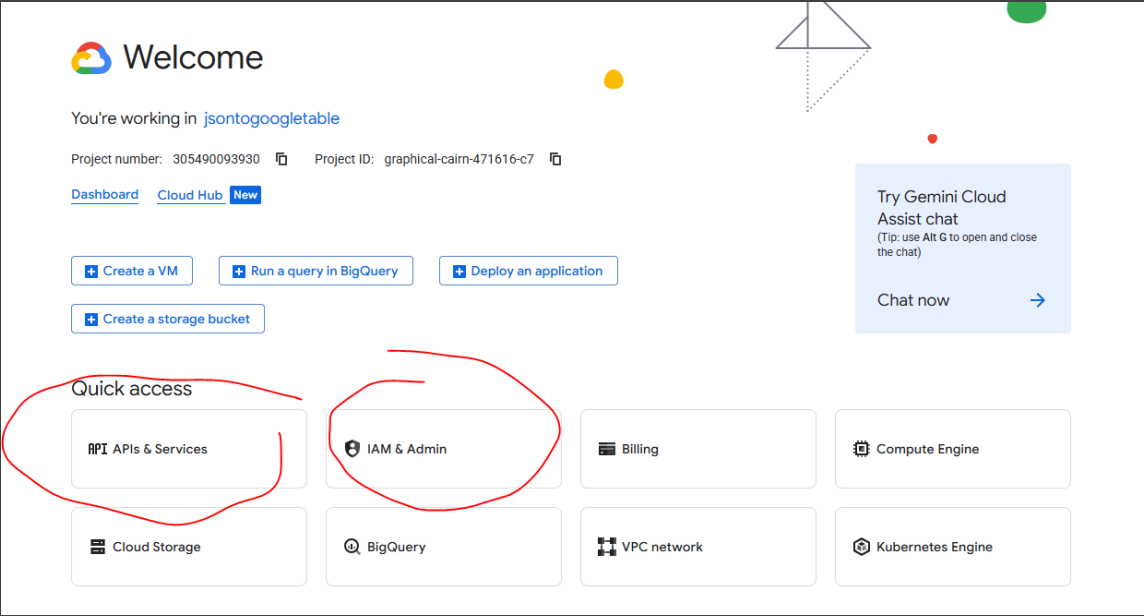
Системы с подкреплением, оптимизирующие свои решения на основе получаемой обратной связи

Эти системы характеризуются **высокой степенью автономности**, способностью к **самоадаптации** и **сложной инфраструктурой мониторинга**. Они требуют значительных вычислительных ресурсов и комплексных механизмов контроля качества.

Как Антон Юрьевич писал парсер для moodle

Чтобы получить файл `GOOGLE_SHEETS_CREDENTIALS_FILE` (JSON-файл с учетными данными сервисного аккаунта), вам нужно выполнить следующие шаги:

- 1. Создайте проект в Google Cloud Console:**
Перейдите на сайт [Google Cloud Console](#).
- 2. Включите необходимые API:**
 - Откройте меню и перейдите в раздел "APIs & Services" -> "Library" 10.
 - Найдите и включите **Google Sheets API** 10.
 - Также рекомендуется включить **Google Drive API**, так как доступ к таблицам часто требует его.
- 3. Создайте сервисный аккаунт:**
 - Перейдите в "APIs & Services" -> "Credentials" 4.
 - Нажмите "Create Credentials" и выберите "Service account" 4.
 - Задайте имя сервисному аккаунту и нажмите "Create".
- 4. Создайте ключ для сервисного аккаунта:**
 - После создания сервисного аккаунта, найдите его в списке и нажмите на него.
 - Перейдите на вкладку "Keys".
 - Нажмите "Add Key" -> "Create new key".
 - Выберите тип ключа **JSON** 3.
 - Нажмите "Create". JSON-файл с ключом автоматически скачается на ваш компьютер 3.



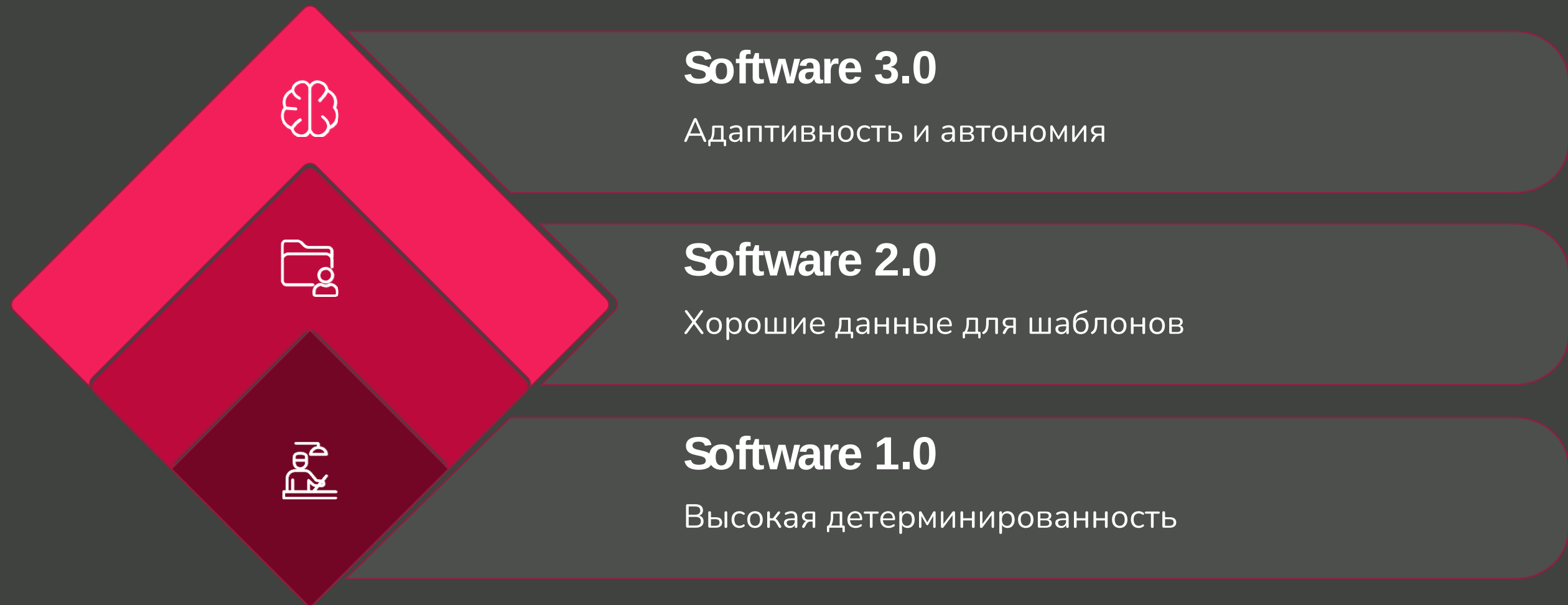
https://gitverse.ru/dayekb/moodle_to_googlesheets

Гибридные системы

Большинство современных промышленных решений сочетают элементы всех трёх парадигм:

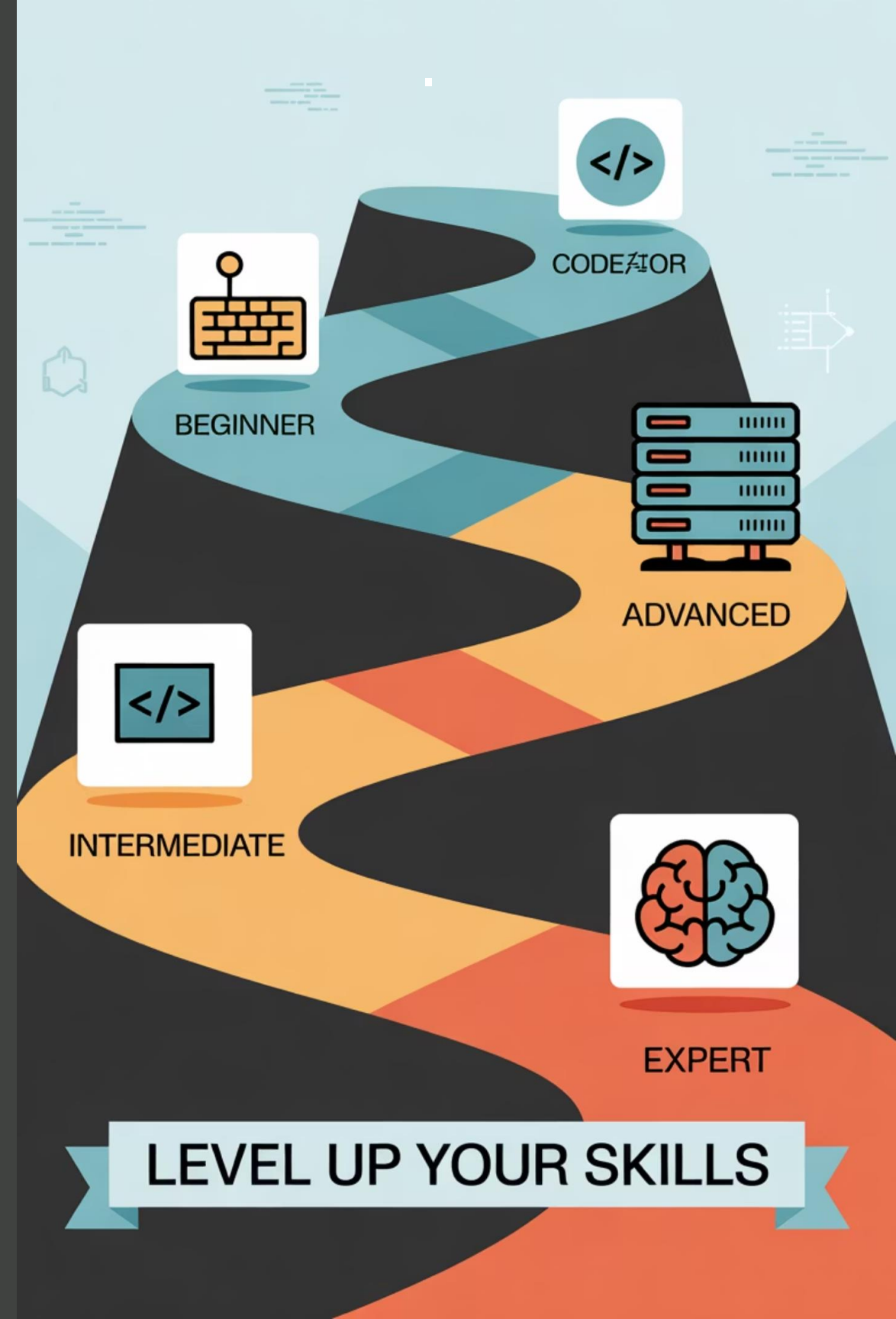
NVIDIA Triton Inference	Оптимизирует выполнение ML-моделей (2.0) с помощью детерминированного кода (1.0) и автоматической адаптации ресурсов (3.0)
ML-оптимизированные БД	BigQuery ML сочетает классические алгоритмы SQL (1.0) с возможностями обучения моделей (2.0) и автоматической оптимизацией запросов (3.0)
Dell EMC с AutoAI	Инфраструктурные решения (1.0) с поддержкой автоматического обучения (2.0) и динамической оптимизации ресурсов (3.0)
AWS SageMaker Pipelines	Комбинирует DevOps-инструменты (1.0) с ML-фреймворками (2.0) и автоматическим масштабированием ресурсов (3.0)

Применимость парадигм



Выбор парадигмы зависит от **характера решаемой задачи**, доступных **данных** и требований к **прозрачности решений**.

Часть 5: Рекомендации





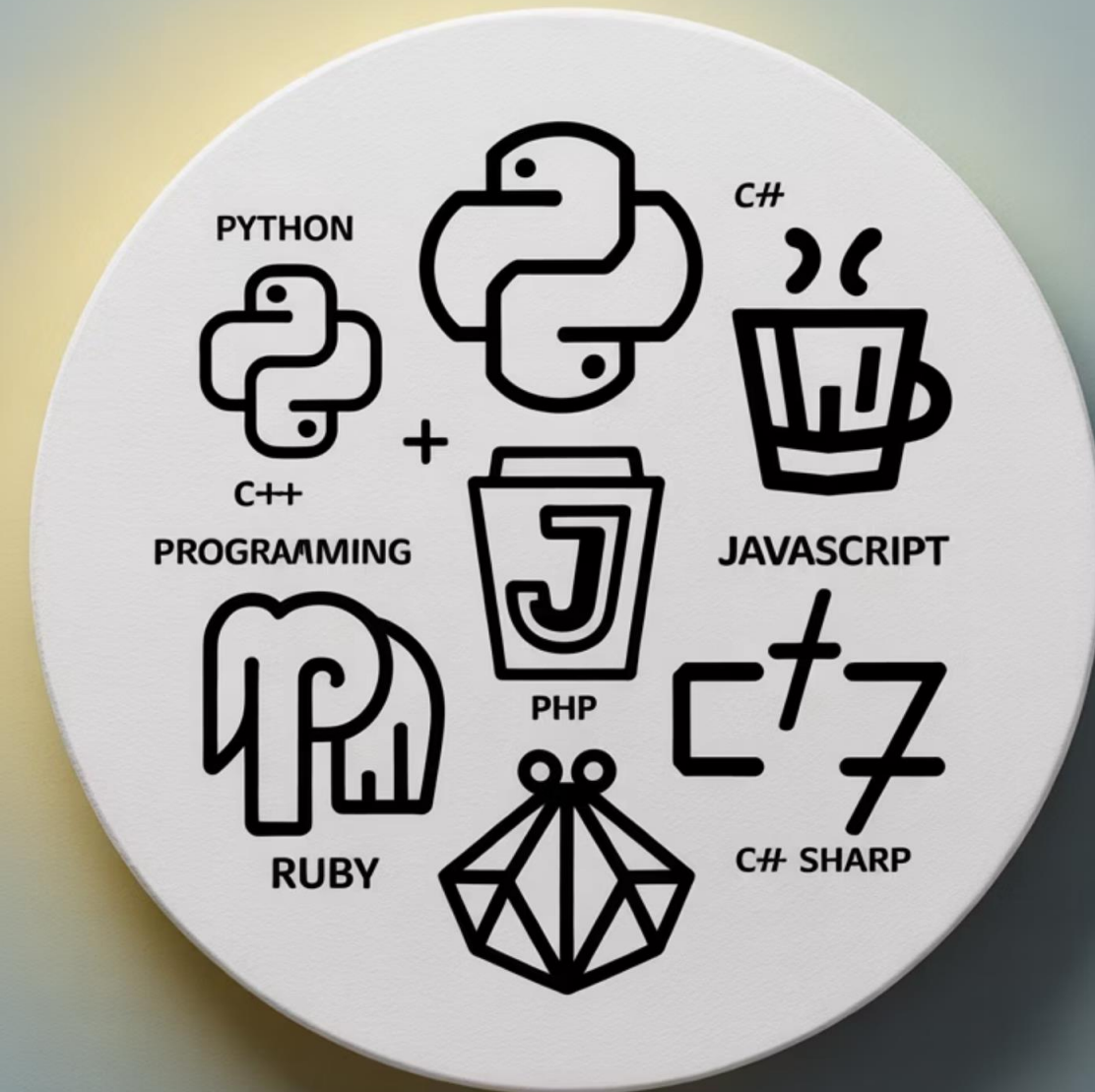
Важность знания различных языков программирования для разных парадигм

Каждая эпоха развития программного обеспечения требует от разработчиков освоения специфических языков и технологий. От традиционных языков программирования до современных фреймворков машинного обучения и новых форматов структурирования промптов — понимание подходящих инструментов для каждой парадигмы становится критически важным навыком современного разработчика.

Software 1.0

Фундаментальные языки программирования

Традиционное программирование остается основой современной разработки. Понимание классических языков и парадигм программирования обеспечивает прочный фундамент для работы с любыми технологиями будущего.



Языковые экосистемы и специализация

Высокий спрос

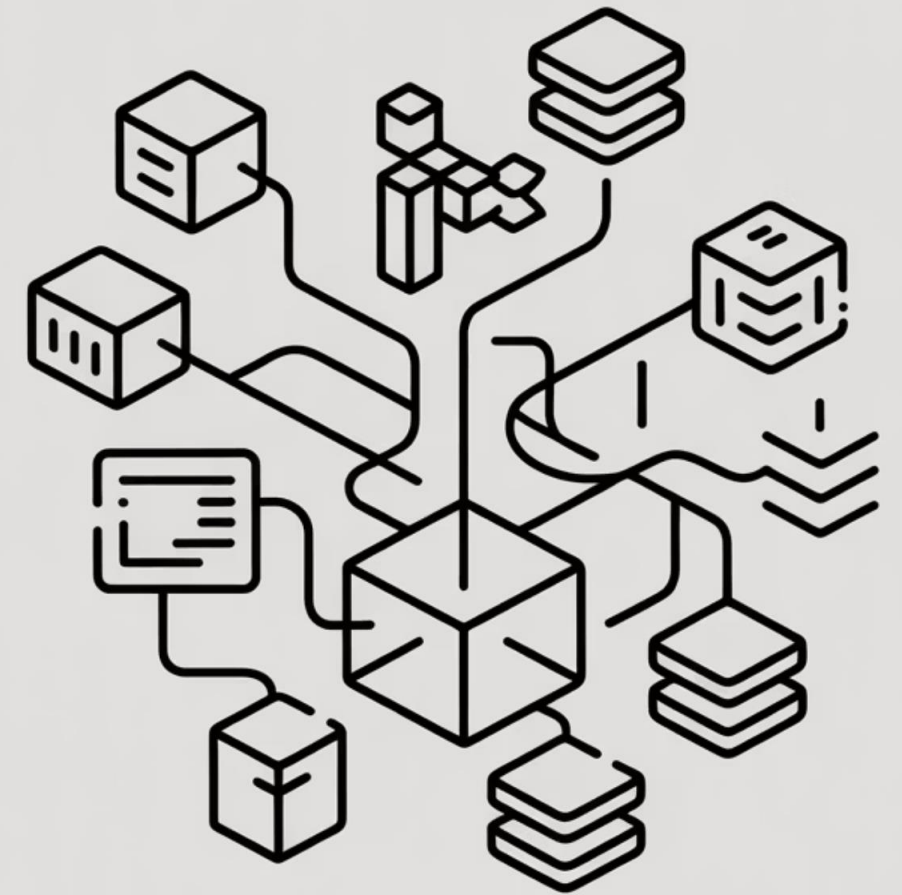
- JavaScript/TypeScript — Веб-платформы, фронтенд и бэкенд
- Python — Data Science, ML, автоматизация
- Java — Enterprise-приложения, Android
- Go — Микросервисы, облачные технологии

Премиальные ниши

- C++ — Системное программирование, игры
- Rust — Безопасное системное программирование
- Scala — Big Data, функциональное программирование
- Kotlin — Android, серверная разработка

А ещё есть 1С

PROGRAMMING LANGUAGE ECOSYSTEM



Рекомендуемые компетенции: Software 1.0

Базовые знания и навыки:

- Алгоритмы и структуры данных
- Паттерны проектирования
- CI/CD и автоматизация тестирования
- Системное программирование

Ключевые компетенции:

- алгоритмическое мышление,
- оптимизация производительности,
- архитектурное проектирование.

Языки и инструменты:

- C/C++, Java, Rust, Go
- SQL и NoSQL базы данных
- Git, Docker, Jenkins
- Инструменты статического анализа

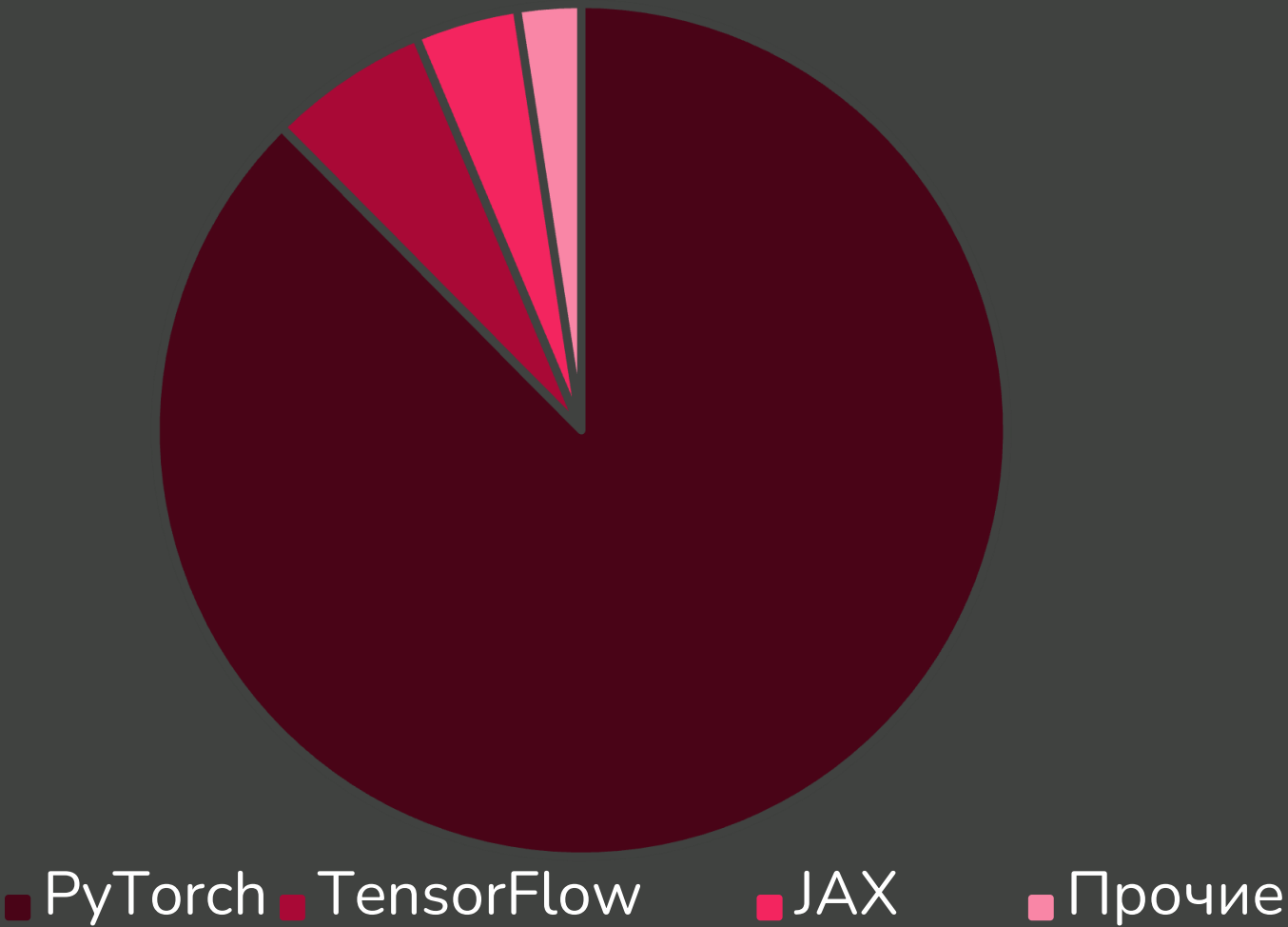


Software 2.0

Эра фреймворков машинного обучения

Революция машинного обучения изменила подход к программированию. Вместо явного написания алгоритмов разработчики создают системы, которые обучаются на данных и самостоятельно выявляют закономерности для решения задач.

Доминирование PyTorch в исследованиях



Статистика моделей на Hugging Face Hub демонстрирует подавляющее преимущество PyTorch в сообществе исследователей и разработчиков ML. Это связано с более интуитивным API и динамической природой фреймворка.

Рекомендуемые компетенции: Software 2.0

Базовые знания и навыки:

- Глубокое обучение и нейросети
- Обработка данных (ETL)
- Оптимизация моделей
- MLOps и автоматизация ML

Языки и инструменты:

- Python, R
- TensorFlow, PyTorch, Scikit-learn
- Apache Spark, Hadoop
- Kubernetes для ML-сервисов

Ключевые компетенции:

- **работа с данными,**
- **статистический анализ,**
- **валидация моделей.**

Software 1.0 → 2.0: Переход к ML

Математические основы

Линейная алгебра, статистика, теория вероятности. Понимание математических принципов, лежащих в основе алгоритмов машинного обучения и нейронных сетей.

Работа с большими данными

ETL процессы, распределенные вычисления, оптимизация пайплайнов обучения. Навыки работы с терабайтами данных и облачными ML-платформами.

Архитектуры нейросетей

От персептронов к трансформерам. Изучение различных архитектур, их применимости к конкретным задачам и принципов работы современных deep learning моделей.

Обучаемый код

Переход от явного к обучаемому коду. Вместо написания алгоритмов создание систем, которые самостоятельно находят оптимальные решения на основе данных.



Software 3.0

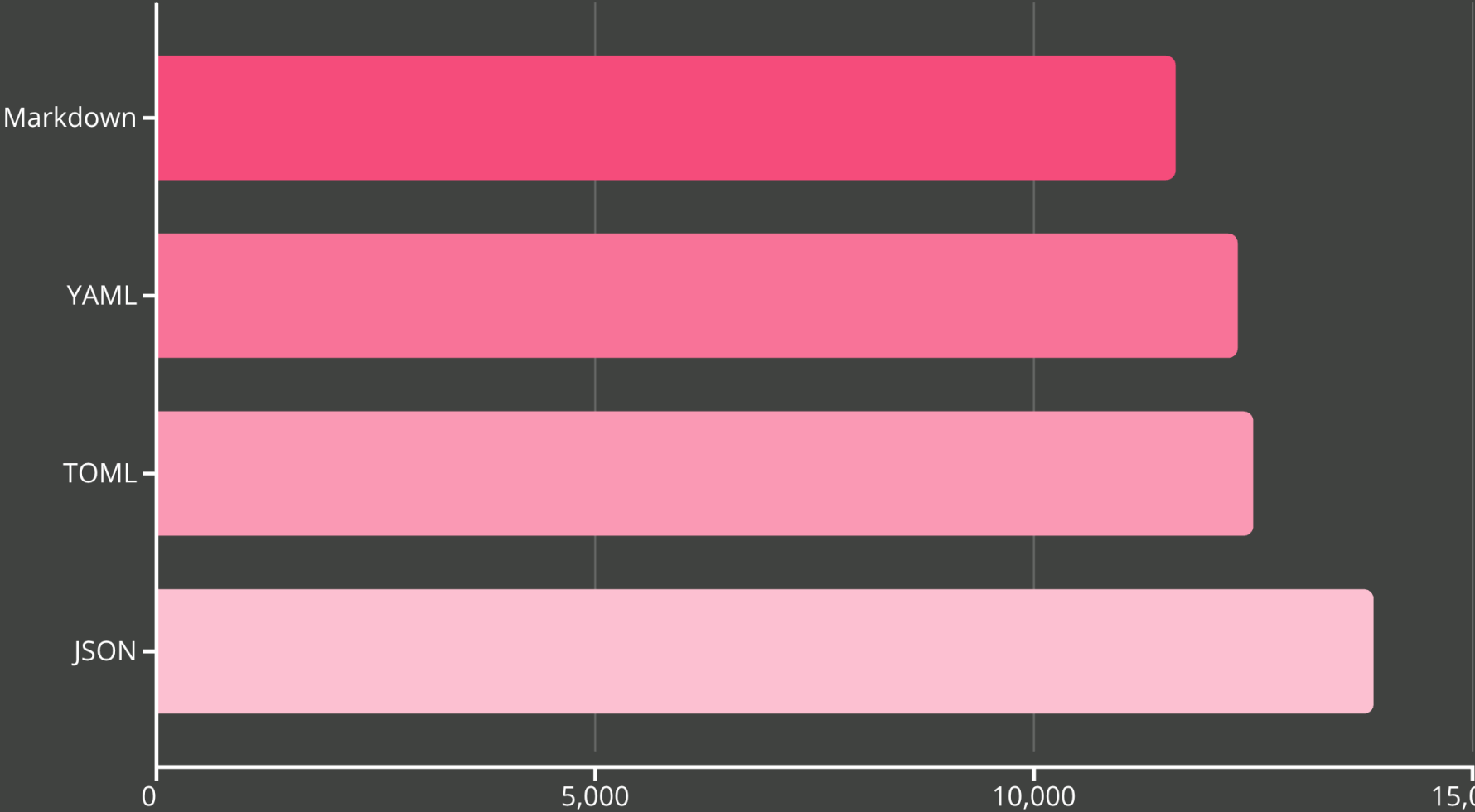
Языки структурирования промптов

Новая парадигма программирования, где код пишется на естественном языке, а AI-системы интерпретируют намерения человека. Эффективность промпт-инженеринга становится критически важной для работы с LLM и созданием AI-агентов.

Форматы для различных задач

Формат	Лучше для	Ключевые преимущества	Ограничения
Markdown	Читаемые промпты	Простота чтения/записи, токен-эффективность	Нужны спец. инструменты парсинга
JSON	Сложные данные	API-стандарт, широкая интеграция	Сложность чтения человеком
YAML	Настройки, конфигурации	Читаемость, поддержка комментариев	Чувствительность к отступам
CSV	Табличные данные	Максимальная компактность	Нет продвинутых функций

Эффективность форматов данных



Markdown экономит 15-20% токенов по сравнению с JSON, что критично для масштабных промпт-систем. Меньшее количество токенов означает снижение затрат на API и увеличение контекстного окна для полезной информации.

Рекомендуемые компетенции: Software 3.0

AutoML и метаобучение

Понимание принципов автоматического выбора моделей и гиперпараметров

Human-in-the-loop

Проектирование систем с эффективным включением человека в процесс

Мониторинг моделей

Разработка систем наблюдения за качеством и производительностью моделей

Безопасность AI

Предотвращение негативных последствий автономной работы систем

Этика AI

Понимание этических аспектов и социальных последствий автономных систем

Ключевые компетенции:

- **системное мышление,**
- **управление рисками,**
- **проектирование отказоустойчивых систем.**

Software 2.0 → 3.0: Промпт-программирование

Промпт-инжиниринг

Освоение техник создания эффективных промптов. Few-shot learning, chain-of-thought reasoning, prompt chaining для сложных задач.

1

Vibe Coding

Программирование намерениями вместо точных инструкций. Искусство передачи контекста и желаемого результата через естественный язык.

2**3****4**

Понимание LLM

Изучение возможностей и ограничений больших языковых моделей. Токенизация, attention mechanisms, emergent abilities современных AI-систем.

Агентные системы

Проектирование multi-agent систем с LLM. Координация множественных AI-агентов для решения сложных, многоэтапных задач.

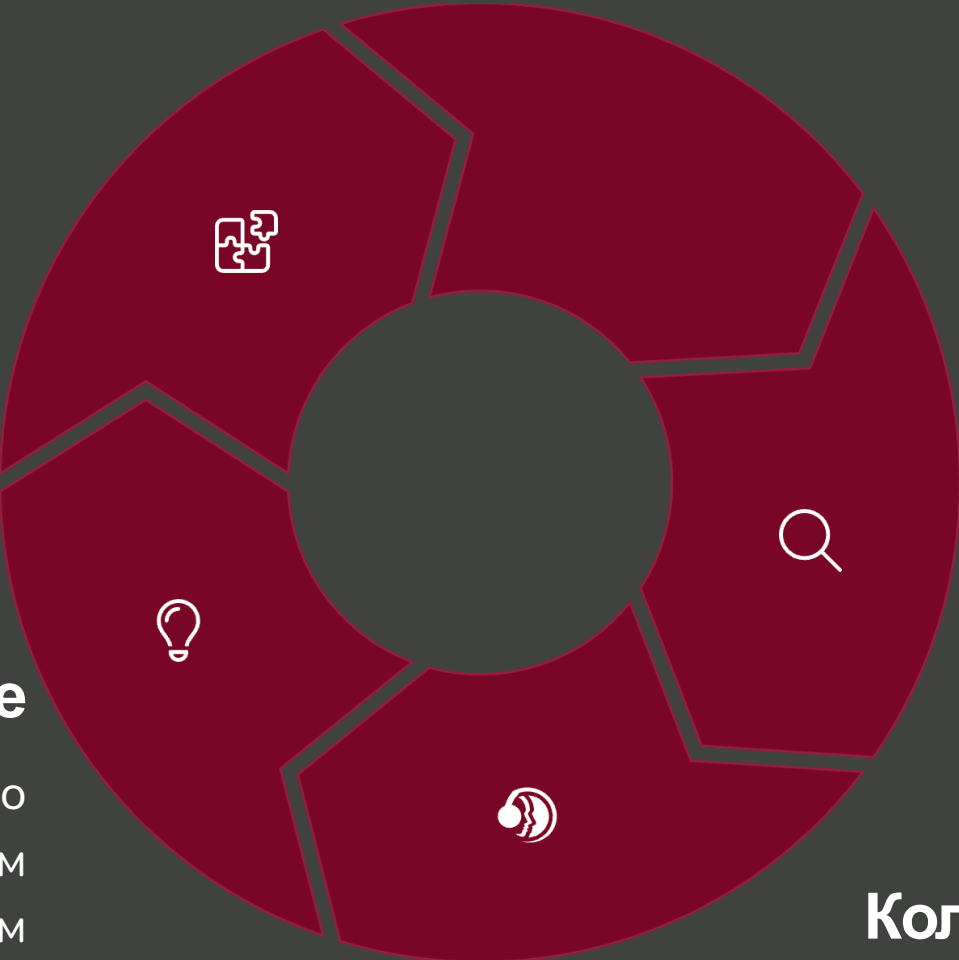
Универсальные навыки программиста

Алгоритмическое мышление

Способность разбивать сложные задачи на простые компоненты независимо от языка программирования.

Непрерывное обучение

Способность быстро адаптироваться к новым технологиям и парадигмам программирования.



Системный дизайн

Проектирование масштабируемых архитектур, понимание trade-offs и принципов построения надежных систем.

Отладка и профилирование

Навыки поиска и устранения багов, оптимизации производительности кода в любой среде разработки.

Коллаборация

Работа в команде, code review, документирование, знание Git и современных практик разработки.

Рекомендации по выбору языка

1

Определите цель

Веб-разработка, мобильные приложения, системное программирование, data science — каждая область имеет свои предпочтительные инструменты и экосистемы.

2

Изучите рынок труда

Анализируйте вакансии в вашем регионе и желаемых компаниях. Спрос на языки может сильно варьироваться географически и по индустриям.

3

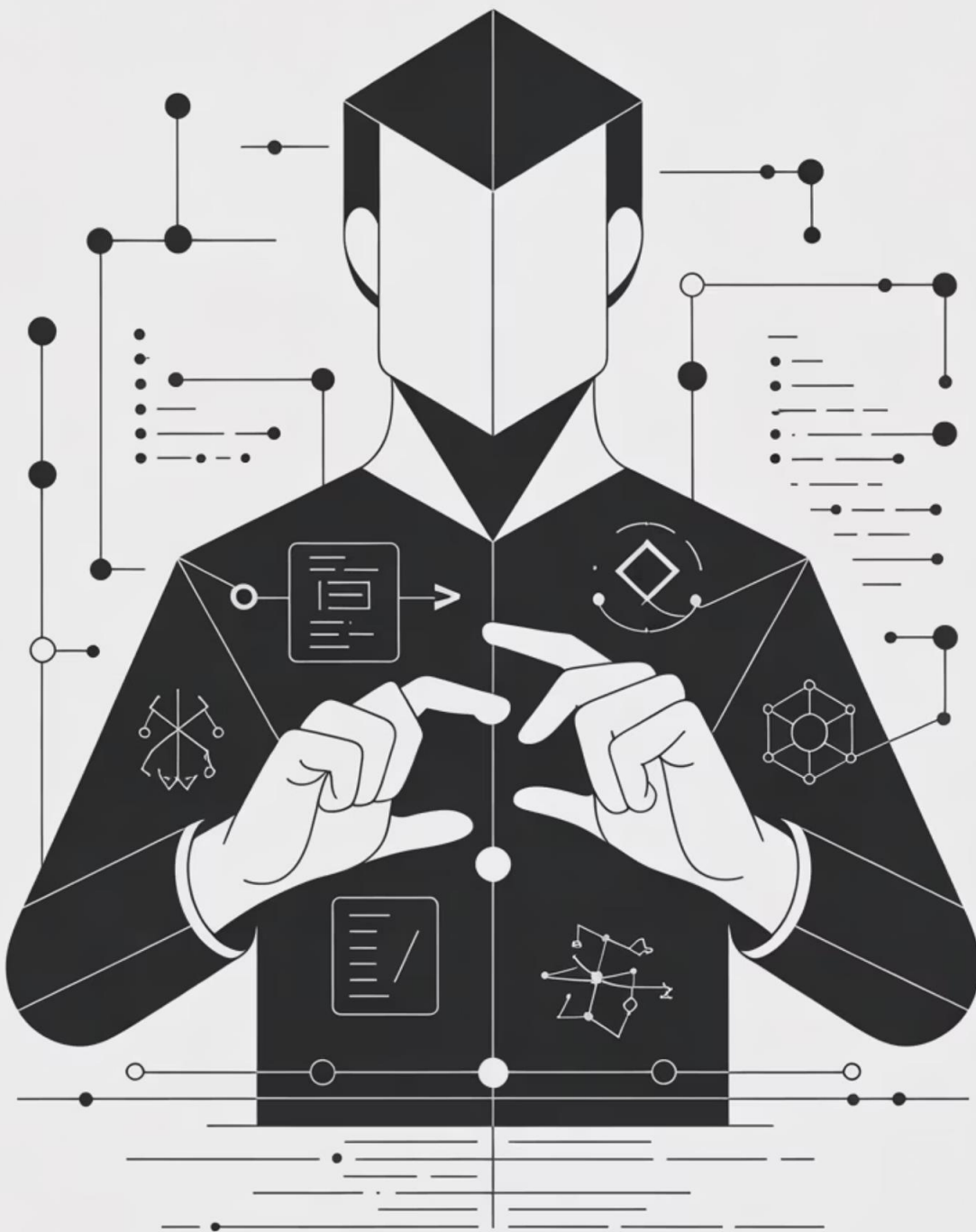
Учитывайте экосистему

Библиотеки, фреймворки, инструменты разработки, community support — все это влияет на продуктивность и карьерные перспективы.

4

Планируйте эволюцию

Выбирайте язык, который обеспечивает естественный путь к смежным технологиям и новым парадигмам программирования.



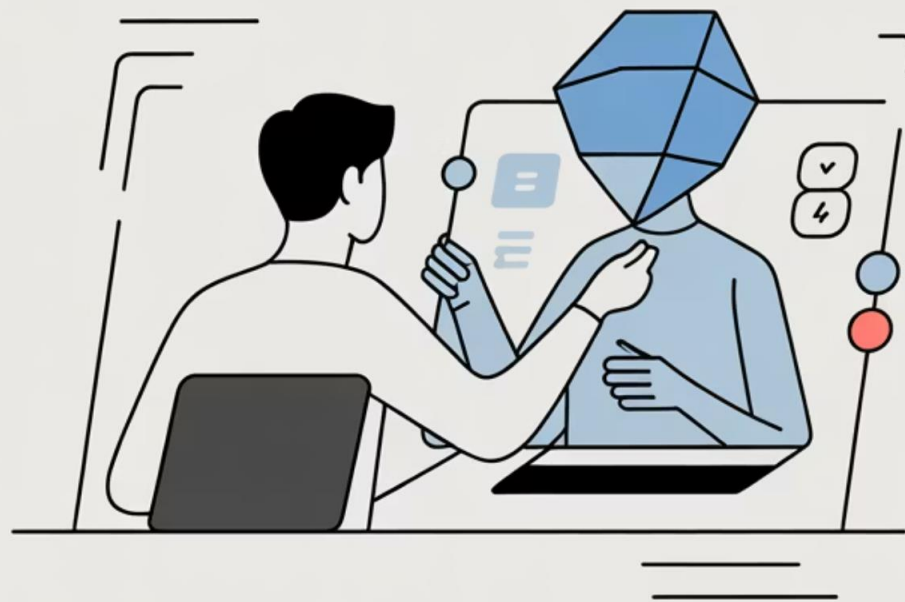
Заключение: Мультипарадигматический разработчик

Эволюция навыков программирования отражает фундаментальные изменения в том, как мы создаем программное обеспечение.

- В эпоху Software 1.0 критичными были знания конкретных языков программирования и алгоритмов.
- Software 2.0 добавил требования понимания фреймворков машинного обучения и архитектур нейросетей.
- Software 3.0 вводит новое измерение — способность эффективно коммуницировать с AI через структурированные промпты и естественный язык.

Современный разработчик должен владеть инструментами из всех трех парадигм: традиционными языками для системной логики, ML-фреймворками для обучаемых компонентов, и техниками промпт-инженеринга для интеграции с AI-системами.

Accelerate your development



Долганов Антон Юрьевич

Будущее за теми, кто адаптируется

Успех в многопарадигматической среде требует не только технических навыков, но и понимания того, **когда и какую парадигму применять** для оптимального решения задач. Инвестиции в изучение принципов, лежащих в основе различных подходов к программированию, окупятся независимо от того, какие конкретные технологии будут доминировать завтра.

Ключевой принцип:



Изучайте не только синтаксис языков, но и философию, стоящую за каждой парадигмой программирования. Это даст вам конкурентное преимущество в любую эпоху развития технологий.

При подготовке презентации использовались

- 1** **Perplexity AI**
Для формирования «обзорного текста»
- 2** **Qwen**
Для построения плана и структуры
- 3** **Gamma.app**
Для генерации нескольких черновых презентаций
- 4** **Долганов Антон Юрьевич**
Который всё это сводил

Спасибо за внимание!

RU Русский

Есть ли вопросы?

Буду рад ответить на ваши вопросы

US English

Any questions?

I'd be happy to answer your questions

FR Français

Des questions?

Je serais ravi de répondre à vos questions

DE Deutsch

Fragen?

Ich beantworte gerne Ihre Fragen

ES Español

¿Preguntas?

Estaré encantado de responder sus preguntas

CN 中文

有问题吗?

我很乐意回答您的问题

Готов к диалогу

Ваши вопросы помогут нам углубить понимание темы и найти новые точки зрения

Благодарю за ваше время и внимание! Жду ваших вопросов и комментариев.